Modeling languages: alpha and omega

João Pedro PEDROSO|DIONÍSIO

Faculty of Sciences, University of Porto

APDIO, 27-MAR-2026

../../papers/cmup-info.png

Outline

The Pre-History: Joseph Fourier (1826)

The Concept of Projection

Before Simplex, Fourier devised a method for solving systems of linear inequalities:

- Method: Fourier–Motzkin Elimination.
- Goal: Determining the feasibility of undetermined systems.
- Insight: Optimization (min-max) is equivalent to solving inequalities.

Comparing "Walking" vs. "Projecting"

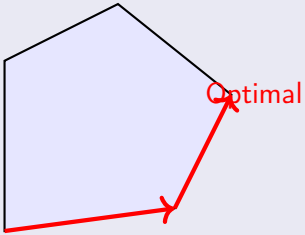
- Fourier (1826): Project the n -dimensional feasible region onto lower dimensions.
  - Problem: Inequality count grows doubly exponentially (2^{2^n}).
- Dantzig (1947): Move between vertices of the n -dimensional region.
  - Advantage: Usually requires only m to $3m$ iterations (where m is the number of constraints).

Historical Note

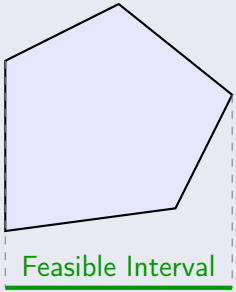
Dantzig noted that had he known FME he might have been discouraged by the perceived

Visualizing the Two Approaches

Vertex Walk vs. Projection



Simplex: Vertex Walk



Fourier: Projection

Why Fourier's Method "Explodes"

- Simplex: Complexity depends on the number of vertices (usually efficient).
- Fourier-Motzkin: Each variable elimination creates a new set of inequalities.
  - If you have m inequalities, eliminating one variable can result in up to $(m/2)^2$ new ones.
  - The Omega of manual methods: Impossible for Stigler's 77 variables, but theoretically vital for proving the Farkas Lemma.

The "Alpha" of Optimization: The Stigler Diet (1947)

The Problem

George Stigler (1945) sought the minimum cost to feed a 154lb man.

- Goal: Minimize daily cost of food.
- Scale: 77 candidate foods, 9 nutritional constraints.

The Mathematical Formulation

- Minimize $Z = \sum_{j=1}^{77} c_j x_j$
- Subject to: $\sum_{j=1}^{77} a_{ij} x_j \geq b_i \quad \forall i \in \{1, \ldots, 9\}, \qquad x_j \geq 0 \quad \forall j \in \{1, \ldots, 77\}$
- Where:
  - c_j: Cost of food j
  - a_{ij}: Amount of nutrient i in food j
  - b_i: RDA for nutrient i (Calories, Protein, Calcium, Iron, Vit A, B1, B2, Niacin, Vit C)

The Human "Computer": Jack Laderman's Feat

Before Machines

In 1947, the "Simplex Method" was brand new (Dantzig). There was no "software."

- Team: 9 human clerks at the National Bureau of Standards.
- Hardware: Hand-operated desk calculators
- Effort: 120 man-days of computation.

Results

- Stigler's heuristic guess: $39.93/year (1939 prices).
- Laderman's Simplex solution: $39.69/year.
- Fun Fact: The "optimal" diet consisted mostly of wheat flour, evaporated milk, cabbage, spinach, and navy beans. Stigler called it the "human dog biscuit."

The First Machine Implementation: SEAC & IBM 701

SEAC (1951) - The First Electronic Solver

- Machine: Standards Eastern Automatic Computer (NBS).
- Implementation: First general-purpose Simplex code for a stored-program computer.
- Language: Machine Code / Binary (no high-level languages like Fortran existed yet).
- Scale: Solved a 48x71 Air Force model in minutes.

IBM 701 (1954) - The First "Commercial" Code

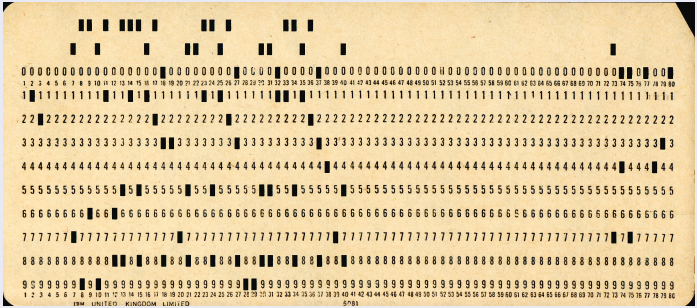
- Developer: William Orchard-Hays at RAND.
- Language: Assembly (highly optimized for the 701's architecture).
- Impact: Could handle ~100 constraints. This code (and the later RSLP1 for IBM 704) moved LP from research to the oil industry (refining/blending).
- The "Omega" Seed: This was the start of Matrix Generators, the tedious precursors to the modeling languages we use today.

Before Modeling Languages: Physical Data Entry

The Punched Card Era (1950s)

In the "Alpha" phase, there was no `model.lp` or `model.py`. Data was physical.

- Storage: IBM 80-column punched cards.
- Method: One non-zero coefficient per card.
- Hardware: IBM 026 Keypunch & 056 Verifier.



Before Modeling Languages: Workflow

1. Mathematical Model: Written on paper.
2. Matrix Tabulation: Coefficients mapped to row/column indices.
3. Keypunching: Manual transcription to cards.
4. Verification: Mechanical "double-checking" of holes.

Technical Bottleneck

- A typo in one card meant the Simplex algorithm would diverge or find a wrong solution.
- There was no "Algebraic" check—only a mechanical one.
- Transition: This pain directly to the creation of Matrix Generators in the 1960s.

References

- Bixby, R. E. (2012). A brief history of linear and mixed-integer programming computation. Documenta Mathematica Series, 107–121. https://doi.org/10.4171/dms/6/16
- Dantzig, G. B. (2020). Impact of linear programming on computer development. Computers in Mathematics, 233–240. https://doi.org/10.1201/9781003072157-6
- Gass, S. I. (2002). The first linear-programming shoppe. Operations Research, 50(1), 61–68. https://doi.org/10.1287/opre.50.1.61.17781

The Representation Gap: Alpha vs. Machine

From Human Thought to Solver Input

Before AMLs (like AMPL or JuMP), there was a "lost in translation" phase. To a mathematician, the model is a set of summations; to the computer, it is a flat list of entries.

Comparison: The Stigler Diet Problem

Format	Representation of $a_{ij}x_j \geq b_i$	Readability
Matrix Generator	`FOR I=1 TO 9: WRITE(6,10) ROW(I), RHS(I)`	Zero
MPS (Machine)	`COL001 NUTRIENT01 0.0034`	
Algebraic (α)	$\sum_{j \in J} a_{ij}x_j \geq b_i, \quad \forall i \in I$	High

The "Middle Ages": Matrix Generators

The "Logic" of 1965

- Instead of writing math, you wrote code to print strings and formatted numbers.

Why this was fragile:

- Formatting: The ‘130 FORMAT‘ statement strictly controls the 80-column spacing. One mistake here makes the MPS file unreadable.
- Maintenance: To add a nutrient, you must change the ‘DATA‘ statements, the ‘DO‘ loop limits, and potentially the ‘FORMAT‘ strings.

The "Middle Ages"

A Matrix Generator (Fortran)

```
C     FORTRAN MATRIX GENERATOR FOR STIGLER DIET
      DATA FOODS/'WHEAT', 'CORNM', 'EVAPM', 'MARGA'/
      DATA NUTRI/'COST ', 'CALOR', 'PROT ', 'CALCI'/

      WRITE(6, 100) 'NAME', 'STIGLER'
      WRITE(6, 110) 'ROWS'
      WRITE(6, 120) ' N ', 'COST'
      DO 10 I=2, 5
   10 WRITE(6, 120) ' G ', NUTRI(I)

      WRITE(6, 110) 'COLUMNS'
      DO 20 J=1, 77
C       INNER LOOP FETCHES DATA FROM ARRAYS AND FORMATS FOR MPS
        WRITE(6, 130) FOODS(J), 'COST', COST(J), 'CALOR', CAL(J)
   20 CONTINUE

  100 FORMAT(A4, 10X, A8)
  110 FORMAT(A)
  120 FORMAT(A3, 2X, A8)
  130 FORMAT(4X, A8, 2X, A8, F12.4, 2X, A8, F12.4)
```

The "Machine" Language: MPS Format

Stigler Diet in 80 Columns

- Before AMLs, this is what a solver "saw." Note the rigid column alignment (Columns 1, 5, 15, 25, 40, 50).

```
NAME          STIGLER
ROWS
 N  COST
 G  CALORIES
 G  PROTEIN
 G  CALCIUM
COLUMNS
    WHEAT     COST            0.036     CALORIES       3.3
    WHEAT     PROTEIN         13.3      CALCIUM        0.9
    CORNMEAL  COST            0.018     CALORIES       3.0
    CORNMEAL  PROTEIN         10.0      CALCIUM        2.0
RHS
    RHS1      CALORIES        3.0       PROTEIN        70.0
    RHS1      CALCALCIUM      0.8
ENDATA
```

Anatomy of a Nightmare

- Fixed Width: If "WHEAT" was shifted one space to the right, the parser would fail.
- Flat Structure: The summation $\sum a_{ij}x_j$ is gone; every (i,j) pair is an individual line.
  - This is exactly the format that was once physically punched into cards.
  - Legacy: Gurobi, CPLEX, ... still read this today for benchmarking.

The "Technical Debt" of the 1970s

- Decoupling: In the algebraic form, data (a_{ij}) is separate from logic ($\sum$).
- The MPS Nightmare: In MPS, data and logic are "fused".
- If you add one nutrient to the diet, you must recalculate and re-punch the entire file.
- The Result: Modeling was slow, error-prone, and required "Model Librarians".